# Practice Set 1 of 3 · Questions

Weeks 1–7: loops, digits, lists and functions

## Exercise 1. A Number Sequence [Easy]

ex1.py · for, range(start, stop, step)

Write a **for** loop that prints the sequence below on one line. There is no input.

EXPECTED OUTPUT

```
-3 11 25 39 53 67 81
```

## Exercise 2. Zero the Negatives [Easy]

ex2.py · def, range(len(list)), changing a list in place

Write a function `zero_negatives(numbers)` that replaces every negative number in the list with 0. The function changes the list it receives and returns nothing.

EXAMPLE

```
data = [3, -2, 5, -8, 0]
zero_negatives(data)
print(data)
```

OUTPUT

```
[3, 0, 5, 0, 0]
```

## Exercise 3. Sum of Odd Numbers [Easy]

ex3.py · input, for, range with step, accumulator

Read a whole number **N** (N > 0). Print the sum of all odd numbers from 1 to N (N included if it is odd).

EXAMPLE RUN 1

```
N: 7
16
```

EXAMPLE RUN 2

```
N: 10
25
```

## Exercise 4. Fixed Points [Easy]

ex4.py · def, range(len(list)), counting

Write a function `count_fixed_points(numbers)` that returns how many positions i have `numbers[i] == i`.

EXAMPLE

```
print(count_fixed_points([0, 2, 2, 3]))
print(count_fixed_points([-1, 1, 2, 4, 3]))
```

OUTPUT

```
3
2
```

## Exercise 5. Digits to a Number [Medium]

ex5.py · input in a loop, building a number with * 10

Read a whole number **N**, then N digits (0 to 9) one by one. Print the number that these digits form when written side by side. Use only the operators + * %; do not use strings or **.

EXAMPLE RUN

```
How many digits? 5
Digit: 2
Digit: 3
Digit: 4
Digit: 5
Digit: 6
23456
```

## Exercise 6. Running Sum [Medium]

ex6.py · def, accumulator, building a new list

Write a function `running_sum(numbers)` that returns a new list where element *i* is the sum of the first *i* + 1 numbers.

EXAMPLE

```
print(running_sum([1, 2, 3, 4]))
print(running_sum([]))
```

OUTPUT

```
[1, 3, 6, 10]
[]
```

## Exercise 7. Sum of Two Squares [Medium]

ex7.py · nested loops, flag variable

Read a whole number **N** (N > 0). If N can be written as $a^2 + b^2$ with positive whole numbers a and b, print every pair a b on its own line. Otherwise print 0.

EXAMPLE RUN 1

```
N: 40
2 6
6 2
```

EXAMPLE RUN 2

```
N: 51
0
```

EXAMPLE RUN 3

```
N: 50
1 7
5 5
7 1
```

## Exercise 8. Remove Consecutive Duplicates [Medium]

ex8.py · def, comparing with the last element of the new list

Write a function `squeeze(numbers)` that returns a new list where each run of equal neighbouring numbers is replaced by one number.

EXAMPLE

```
print(squeeze([1, 1, 2, 2, 2, 3, 1, 1]))
print(squeeze([]))
```

OUTPUT

```
[1, 2, 3, 1]
[]
```

## Exercise 9. Rotate Right [Medium]

ex9.py · def, slicing, %

Write a function `rotate_right(numbers, k)` that returns a new list rotated to the right by k positions. k may be larger than the length of the list.

EXAMPLE

```
print(rotate_right([1, 2, 3, 4, 5], 2))
print(rotate_right([1, 2, 3, 4, 5], 7))
print(rotate_right([1, 2, 3], 0))
```

OUTPUT

```
[4, 5, 1, 2, 3]
[4, 5, 1, 2, 3]
[1, 2, 3]
```

## Exercise 10. Min and Max So Far [Medium]

ex10.py · def, returning two values, searching

Write a function `running_min_max(numbers)` that returns two lists: the minimum so far and the maximum so far, after each element. Do not use `min()` or `max()`.

EXAMPLE

```
mins, maxs = running_min_max([5, 2, 7, 1, 6])
print(mins)
print(maxs)
```

OUTPUT

```
[5, 2, 2, 1, 1]
[5, 5, 7, 7, 7]
```

## Exercise 11. IKI + IKI = DORT [Hard]

ex11.py · nested loops, digit extraction, checking that digits are different

In **IKI + IKI = DORT** every letter stands for a digit, and different letters stand for different digits (I and D cannot be 0). Print every pair `IKI DORT` that makes the sum correct, one per line. There is no input.

EXPECTED OUTPUT

```
676 1352
686 1372
767 1534
```

## Exercise 12. Merge Two Sorted Lists [Hard]

ex12.py · def, two indices, while

Write a function `merge(a, b)` that takes two sorted lists and returns one sorted list with all elements. Do not use `sort()`, `sorted()` or `extend()`.

EXAMPLE

```
print(merge([1, 3, 5], [2, 4, 6]))
print(merge([], [1, 2]))
print(merge([1, 1, 4], [1, 2]))
```

OUTPUT

```
[1, 2, 3, 4, 5, 6]
[1, 2]
[1, 1, 1, 2, 4]
```

## Exercise 13. Product of the Others [Hard]

ex13.py · def, nested loops or prefix products

Write a function `product_except_self(numbers)` that returns a list where element *i* is the product of all the other elements. Do not use division.

EXAMPLE

```
print(product_except_self([1, 2, 3, 4]))
print(product_except_self([0, 5, 2]))
```

OUTPUT

```
[24, 12, 8, 6]
[10, 0, 0]
```

## Exercise 14. Last One at the Table [Hard]

ex14.py · def, pop(i), %, while

N people sit around a round table, numbered 1 to N clockwise. Starting at person 1, count K people (person 1 counts as the first); the K-th person leaves the table. Counting continues from the next person, and so on until the table is empty. Write a function `leaving_order(n, k)` that returns the numbers in the order in which people leave.

EXAMPLE

```
print(leaving_order(5, 2))
print(leaving_order(10, 3))
```

OUTPUT

```
[2, 4, 1, 5, 3]
[3, 6, 9, 2, 7, 1, 8, 5, 10, 4]
```

## Exercise 15. Matrix Tools [Hard]

ex15.py · def, 2D lists, nested loops, no zip

A matrix is a list of rows. Write two functions:

1. `transpose(matrix)`: returns a new matrix where rows and columns are swapped. Do not use `zip`.
2. `diagonal_sum(matrix)`: for a square matrix returns the sum of both diagonals; the centre element of an odd-sized matrix is counted once.

EXAMPLE

```
print(transpose([[1, 2, 3], [4, 5, 6]]))
print(diagonal_sum([[1, 2, 3], [4, 5, 6], [7, 8, 9]]))
print(diagonal_sum([[1, 2], [3, 4]]))
```

OUTPUT

```
[[1, 4], [2, 5], [3, 6]]
25
10
```

# Hints

Try each exercise on your own first. Look here only when you are stuck.

**Exercise 1. A Number Sequence**

Look at the difference between two neighbours. range() can take a third value, the step. Remember that the stop value is never included.

**Exercise 2. Zero the Negatives**

`for x in numbers: x = 0` does not change the list. You need the index.

**Exercise 3. Sum of Odd Numbers**

Start a total at 0 before the loop, add inside the loop, print after the loop.

**Exercise 4. Fixed Points**

You need the index, so loop over `range(len(numbers))`.

**Exercise 5. Digits to a Number**

If the number so far is 23 and the next digit is 4, the new number is 23 × 10 + 4.

**Exercise 6. Running Sum**

Keep a total, add the next number, and append the total to the result.

**Exercise 7. Sum of Two Squares**

Try every a and every b from 1 upwards. You can stop a loop once its square is bigger than N.

**Exercise 8. Remove Consecutive Duplicates**

Compare the current number with the last element of the result, `result[-1]`. The first number is always kept.

**Exercise 9. Rotate Right**

Use `k % len(numbers)` first. Then the last k elements go to the front: `numbers[-k:] + numbers[:-k]` (careful when k is 0).

**Exercise 10. Min and Max So Far**

A function can return two values (Week 7 slide 11): `return mins, maxs`.

**Exercise 11. IKI + IKI = DORT**

Only I and K are free: try all of them, compute DORT = 2 × IKI, and take its four digits apart. Then check that the six digits I, K, D, O, R, T are all different.

**Exercise 12. Merge Two Sorted Lists**

Keep one index per list. Take the smaller of the two current elements. When one list is finished, copy the rest of the other.

**Exercise 13. Product of the Others**

Simplest: for every i, multiply all elements with a different index. Harder: one pass from the left for the products before i, one from the right for those after i.

**Exercise 14. Last One at the Table**

Keep the people in a list and the position where counting starts. After `pop(i)` the next person sits at the same index. Use `%` to wrap around.

**Exercise 15. Matrix Tools**

Element `matrix[i][j]` becomes element `[j][i]` of the transpose. A 2×3 matrix becomes a 3×2 matrix.